Giải Sudoku tổng quát bằng SAT: so sánh ba mã hóa At-Most-One và các đối chứng CP/ILP

Tạ Đình Nguyên

INT3301E_4, Mã sinh viên: 24021587,

Tóm tắt

Nghiên cứu này biểu diễn Sudoku tổng quát bằng mô hình Boolean ô–giá trị và so sánh ba mã hóa At-Most-One: pairwise, sequential counter và bitwise, cùng Glucose3. Bộ dữ liệu gồm 60 đề synthetic ở bốn kích thước 4, 9, 16, 25 và ba mật độ clue, cộng 20 đề public 9×9. CP-SAT/HiGHS được thử trên tập con 16 đề; Gurobi, CPLEX MIP và CP Optimizer trên tập con 10 đề. Với ba lần lặp và cutoff 10 giây gồm dựng/nạp mô hình và giải, 906 lượt cho 903 nghiệm được checker độc lập chấp nhận và ba timeout HiGHS trên cùng một đề. Cả ba mã hóa SAT giải đủ 80 đề trong mọi lần lặp. Trên năm đề synthetic 25×25, 30% clues, median total của pairwise, sequential và bitwise lần lượt là 0.402449, 0.159527 và 0.210650 giây; chi phí build lớn hơn solve. Tuy nhiên, thứ tự thay đổi theo kích thước; CPLEX MIP thấp hơn pairwise trên bốn đề public chung. Kết quả nhấn mạnh việc tính cả chi phí dựng mô hình và so đúng tập giao, không chứng minh một solver hoặc mã hóa thắng trên mọi Sudoku. Uniqueness chưa được kiểm và độ phủ đối chứng còn nhỏ.

Sinh viên Tạ Đình Nguyên, mã số 24021587, lớp/mã môn INT3301E_4. Giảng viên: TS. Tô Vân Khánh. Hạn nộp đã xác nhận: 23:59 ngày 19/10/2026, giờ Việt Nam.

1. Giới thiệu và nghiên cứu liên quan

Sudoku có luật ngắn gọn nhưng cho phép biểu diễn cùng một bài toán khả thi bằng SAT, CP hoặc ILP. Khi tăng kích thước bảng, số biến, ràng buộc và chi phí dựng mô hình có thể khác nhau giữa các biểu diễn. Vì vậy, đánh giá một cách giải cần phân biệt kích thước mô hình, thời gian dựng/nạp và thời gian giải; số clause nhỏ hơn tự nó chưa chứng minh solver nhanh hơn. Công việc này xét Sudoku tổng quát với $n \geq 2$, $N = n^2$, ưu tiên ảnh hưởng của ba AMO encoding trong cùng mô hình SAT.

1.1. Cơ sở liên quan

Lynce–Ouaknine trình bày biểu diễn Sudoku bằng biến Boolean ô–giá trị và phân biệt mô hình minimal/extended [11]. Bài này là nền tảng mô hình, không cung cấp kết quả cho bộ dữ liệu của dự án. Sinz trình bày sequential unary counter cho ràng buộc cardinality [14]. PySAT cung cấp toolkit và API CardEnc cho các lựa chọn AMO [10, 12]. Dự án giữ cùng ALO/bốn họ EXO, chỉ thay AMO pairwise, seqcounter hoặc bitwise; không đồng nhất bitwise AMO với binary counter trong bài Sinz. Những nguồn này hỗ trợ cách xây dựng, không thay thế kiểm đúng hoặc phép đo của chương trình hiện tại.

1.2. Câu hỏi nghiên cứu

1. Ba AMO encoding ảnh hưởng thế nào đến số biến/phụ, clause và thời gian build/solve/total khi thay kích thước bảng và mật độ clue?
2. Trên đúng tập đề giao nhau, SAT so với CP-SAT và ILP qua HiGHS thế nào về tỷ lệ tìm nghiệm hợp lệ và thời gian trong cùng cutoff?
3. Các đối chứng Gurobi/CPLEX khả dụng trên tập nhỏ cung cấp bằng chứng bổ sung nào, và giới hạn runtime/giấy phép làm giảm độ phủ đến đâu?

S80/B16/C10 chỉ định mẫu so sánh trước phép đo. Mục 9 trả lời các câu hỏi bằng số liệu đã kiểm, luôn lọc đúng tập giao; không ghép thống kê SAT toàn S80 với baseline B16/C10 như cùng một mẫu.

1.3. Phạm vi đóng góp

Đóng góp ở mức bài tập lớn là triển khai mô hình Sudoku tổng quát, tích hợp các encoding đã biết dưới cùng một đặc tả, kiểm chứng mô hình/nghiệm và đánh giá tái lập qua 906 lượt trên các tập so sánh xác định. Công việc không đề xuất encoding mới, không xem runtime/library sẵn có là thuật toán do tác giả phát minh, và không tuyên bố ưu thế trên mọi n hoặc mọi Sudoku.

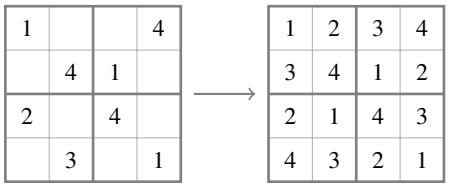
2. Bài toán và ký hiệu

Cho số nguyên $n \geq 2$, đặt $N = n^2$. Sudoku là bảng $N \times N$ chia thành N khối $n \times n$. Hàng và cột được đánh số từ 0; giá trị ô thuộc $D = \{1, \dots, N\}$. Input dùng 0 cho ô trống. Một nghiệm điền đủ bảng, giữ các clues, và chứa mỗi giá trị đúng một lần trong từng hàng, cột và khối. Không yêu cầu nghiệm duy nhất.

Đặt $I = \{0, \dots, N-1\}$, $J = \{0, \dots, n-1\}$ và $G \subseteq I \times I \times D$ là tập các clue khác 0, với $g = |G|$. Biến Boolean $x_{r,c,v}$ đúng khi ô (r, c) mang giá trị v. Có N^3 biến chính. Mapping của chương trình là

$$\mathrm{id}(r, c, v) = (rN + c)N + v \in \{1, \dots, N^3\}. \tag{1}$$

Parser từ chối cấu trúc/miền sai bằng INVALID_INPUT. Clues mâu thuẫn nhưng đúng cấu trúc vẫn được mã hóa; tính không khả thi là UNSAT sau khi được chứng minh. Hình 1 minh họa $n = 2$.



Hình 1: Ví dụ 4×4: input (trái) và một cách hoàn thành (phải) dựng để minh họa; không phải output solver hoặc chứng nhận uniqueness.

3. Mô hình SAT và ba AMO encoding

Cách biểu diễn ô–giá trị và mô hình mở rộng của Lynce–Ouaknine là nền tảng tham khảo [11]. Với nhóm $Z = (z_1, \dots, z_m)$, ràng buộc Exactly-One được tách thành ALO và AMO:

$$\mathrm{EXO}(Z) = \mathrm{ALO}(Z) \wedge \mathrm{AMO}(Z), \qquad \mathrm{ALO}(Z) = \bigvee_{i=1}^{m} z_i. \tag{2}$$

Bốn họ sau, mỗi họ có N^2 nhóm dài N, cùng các clue unit clauses tạo công thức Sudoku:

$$\bigwedge_{r,c \in I} \mathrm{EXO}\{x_{r,c,v} : v \in D\}, \qquad \text{ô;} \tag{3}$$

$$\bigwedge_{r \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : c \in I\}, \qquad \text{hàng–giá trị;} \tag{4}$$

$$\bigwedge_{c \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : r \in I\}, \qquad \text{cột–giá trị;} \tag{5}$$

$$\bigwedge_{p,q \in J, v \in D} \mathrm{EXO}\{x_{pn+u,qn+w,v} : u, w \in J\}, \qquad \text{khối–giá trị;} \tag{6}$$

$$\bigwedge_{(r,c,v) \in G} x_{r,c,v}, \qquad \text{clues.} \tag{7}$$

Như vậy có $4N^2$ nhóm EXO. Code sinh cùng thứ tự nhóm/literal cho mọi encoder; mỗi nhóm thêm ALO trước AMO, cuối cùng thêm clues theo hàng. Không loại clause trùng, không symmetry breaking hay tiền xử lý riêng theo clue.

3.1. Pairwise

$$E_{\mathrm{pw}}(Z) = \bigwedge_{1 \le i < j \le m} (\neg z_i \vee \neg z_j). \tag{8}$$

Mọi cặp input không được cùng đúng. Code tạo trực tiếp các cặp, không dùng biến phụ.

3.2. Sequential counter

Theo nguyên lý sequential unary counter [14], có thể đặc tả AMO bằng chuỗi $s_1, \dots, s_{m-1}$:

$$E_{\mathrm{sc}}(Z, S) = (\neg z_1 \vee s_1) \wedge (\neg z_m \vee \neg s_{m-1}) \wedge \bigwedge_{i=2}^{m-1} \big[(\neg z_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg z_i \vee \neg s_{i-1})\big]. \tag{9}$$

Input đúng làm chuỗi truyền về sau, ngăn một input sau đó cũng đúng. Chiều mở rộng chọn $s_i = \bigvee_{j \le i} z_j$. Chương trình gọi `CardEnc.atmost` với `bound=1`, `EncType.seqcounter` và `top_id` hiện hành [12, 10]. Biểu thức trên mô tả nguyên lý/semantics, không khẳng định thứ tự literal hay biến phụ trùng nguyên byte với binary PySAT.

3.3. Bitwise AMO

Đặt $k = \lceil \log_2 m \rceil$ và gán mỗi input một mã nhị phân khác nhau $b(i) \in \{0,1\}^k$. Với các selector $a_1, \dots, a_k$, đặt $\ell_{ij} = a_j$ nếu $b(i)_j = 1$, và $\ell_{ij} = \neg a_j$ nếu $b(i)_j = 0$. Khi đó

$$E_{\mathrm{bw}}(Z, A) = \bigwedge_{i=1}^{m} \bigwedge_{j=1}^{k} (\neg z_i \vee \ell_{ij}). \tag{10}$$

Hai input đúng đòi hai mã khác nhau nên xung đột ở ít nhất một bit. Khi có một input đúng, chọn mã của nó; khi không có input đúng, chọn selector tùy ý. Chương trình dùng `EncType.bitwise` qua cùng API AMO. Đây là mã hóa AMO cho các literal one-hot, không phải thay biến ô bằng số nhị phân. Nguồn API/source bitwise được ghi riêng [12, 13]; không gán encoding này cho Sinz.

4. Tính đúng đắn

Gọi $F_e(X, A)$ là CNF cho encoding e, gồm cả clues. Mỗi AMO phải thỏa đặc tả $\exists A_Z\, E_e(Z, A_Z) \iff \sum_i z_i \le 1$. Các cấu trúc trên đáp ứng đặc tả này. Trong code, toàn bộ N^3 ID chính được dự trữ trước; mỗi nhóm cấp biến phụ phía trên `top_id` và cập nhật nó, nên các nhóm không dùng chung biến phụ. Khi đó

$$\exists A\, F_e(X, A) \quad \iff \quad \mathrm{Sudoku}_G(X). \tag{11}$$

Chiều thuận. ALO và AMO của ô cho đúng một giá trị trong mỗi ô. Các nhóm hàng–giá trị, cột–giá trị và khối–giá trị cho mỗi giá trị xuất hiện đúng một lần trong các đơn vị tương ứng. Unit clauses giữ mọi clue. Vì vậy giải mã các biến chính của một model CNF tạo một nghiệm hợp lệ.

Chiều ngược. Từ một nghiệm Sudoku giữ clues, đặt đúng các biến ô–giá trị được chọn; mỗi nhóm có đúng một input đúng. Pairwise đã thỏa; sequential mở rộng bằng các OR tiền tố; bitwise chọn mã input đúng. Do các nhóm có tập phụ rời nhau, ghép các mở rộng cục bộ thành một phép gán A thỏa toàn CNF. Không đòi mọi phép gán phụ đều thỏa, cũng không suy nghiệm là duy nhất.

5. Kích thước lý thuyết và số đếm đã kiểm

Với $m \ge 4$, các cấu trúc AMO được trình bày có số phụ $a_e(m)$ và clause $c_e(m)$ như sau:

e	$a_e(m)$	$c_e(m)$
pairwise	0	$m(m-1)/2$
seqcounter	$m-1$	$3m-4$
bitwise	$\lceil \log_2 m \rceil$	$m\lceil \log_2 m \rceil$

(12)

Do mỗi nhóm còn một ALO, tổng lý thuyết là

$$V_e = N^3 + 4N^2 a_e(N), \qquad C_e = 4N^2[1 + c_e(N)] + g. \tag{13}$$

Pairwise có chính xác N^3 biến và $4N^2[1 + N(N-1)/2] + g$ clause khi giữ trùng. Các cấu trúc cho độ lớn clause lần lượt $\Theta(N^4)$, $\Theta(N^3)$, $\Theta(N^3 \log N)$; cả ba có tổng biến $\Theta(N^3)$, vì $g \le N^2$. Đây là kích thước biểu diễn, không phải giới hạn thời gian solve hoặc bằng chứng encoder nào nhanh hơn.

Bảng 1 được sinh từ `diagnostics/model/results.json`: PySAT 1.9.dev16, $N \in \{4, 9, 16, 25\}$. Đây là số đếm CNF trước preprocessing, không phải benchmark; các ca chẩn đoán có $g = 2$ và bảng trừ hai clue unit để hiển thị $C - g$. Seqcounter/bitwise quan sát được khớp các biểu thức trên tại bốn kích thước này. Không suy mọi biến thể/thư viện hay mọi độ dài input có cùng count; API có thể có shortcut ở nhóm nhỏ. Source upstream đã đọc chưa được coi là bằng chứng binary đang chạy.

Bảng 1
Số đếm cấu trúc từ diagnostics; $C - g$ không gồm clue units.

N	AMO	N^3	Phụ	V_e	$C_e - g$
4	pairwise	64	0	64	448
4	seqcounter	64	192	256	576
4	bitwise	64	128	192	576
9	pairwise	729	0	729	11988
9	seqcounter	729	2592	3321	7776
9	bitwise	729	1296	2025	11988
16	pairwise	4096	0	4096	123904
16	seqcounter	4096	15360	19456	46080
16	bitwise	4096	4096	8192	66560
25	pairwise	15625	0	15625	752500
25	seqcounter	15625	60000	75625	180000
25	bitwise	15625	12500	28125	315000

6. Mô hình CP và ILP đối chứng

Các đối chứng mô tả cùng bài toán: tìm một nghiệm giữ clues, không tối ưu độ khó hoặc chất lượng một bảng Sudoku. Phần này đối chiếu các builder đã nghiệm thu; dựng được mô hình không có nghĩa đã giải thành công hay được giấy phép cho phép ở mọi kích thước.

6.1. CP với AllDifferent

Với ký hiệu I, J, D, G đã định nghĩa, đặt một biến nguyên $z_{r,c} \in D$ cho mỗi ô. Ràng buộc là

$$\mathrm{AllDifferent}(z_{r,c} : c \in I), \qquad r \in I, \tag{14}$$

$$\mathrm{AllDifferent}(z_{r,c} : r \in I), \qquad c \in I, \tag{15}$$

$$\text{AllDifferent}(z_{pn+u,qn+w} : u, w \in J), \qquad p, q \in J, \tag{16}$$

$$z_{r,c} = v, \qquad (r, c, v) \in G. \tag{17}$$

Vì mỗi đơn vị có N ô nhận giá trị trong miền N phần tử, AllDifferent buộc mỗi giá trị xuất hiện đúng một lần. Có N^2 biến nguyên và $3N + g$ ràng buộc cấp mô hình; không coi số này là số clause/biến sau biến đổi nội bộ của solver.

Builder CP-SAT tạo `CpModel`, một `CpSolver` mới và đặt một worker [4]. Builder CP Optimizer dùng cùng miền, ràng buộc AllDifferent và clues, với `Workers=1` [9]; executable được kiểm riêng. Các builder chưa thực hiện solve, decode hoặc xác nhận nghiệm trong gói đã bàn giao.

6.2. ILP nhị phân dùng chung

Đặt $y_{r,c,v} \in \{0, 1\}$ cho $r, c \in I$, $v \in D$. Bốn họ đẳng thức và clues là

$$\sum_{v \in D} y_{r,c,v} = 1, \qquad r, c \in I, \tag{18}$$

$$\sum_{c \in I} y_{r,c,v} = 1, \qquad r \in I,\ v \in D, \tag{19}$$

$$\sum_{r \in I} y_{r,c,v} = 1, \qquad c \in I,\ v \in D, \tag{20}$$

$$\sum_{u,w \in J} y_{pn+u,qn+w,v} = 1, \qquad p, q \in J,\ v \in D, \tag{21}$$

$$y_{r,c,v} = 1, \qquad (r, c, v) \in G. \tag{22}$$

Builder xuất ma trận CSR M, vector vế phải toàn 1 và objective toàn 0:

$$\min\ \mathbf{0}^{\mathsf{T}} Y = 0 \quad \text{subject to} \quad MY = \mathbf{1}, \quad 0 \le Y \le 1, \quad Y \in \mathbb{Z}^{N^3}. \tag{23}$$

Do mọi nghiệm hợp lệ có cùng objective 0, đây là bài toán khả thi. Không tối thiểu hóa thời gian chạy bằng objective này. Cột zero-based là $\text{id}(r, c, v) - 1$. Mô hình có N^3 biến nhị phân, $4N^2 + g$ hàng đẳng thức và $4N^3 + g$ phần tử khác 0, trước mọi preprocessing. Mỗi nhóm EXO/clue tạo một hàng; giữ cả các hàng có thể dư thừa, không giảm miền hoặc pruning riêng theo backend.

Cùng cấu trúc ILP được dùng cho các adapter Gurobi, CPLEX MIP và HiGHS. Luồng thực thi, thiết lập luồng/ngân sách, trạng thái và checker được mô tả riêng ở Mục 7; bằng chứng builder không tự chứng minh độ phủ runtime. Về mô hình, liên hệ $y_{r,c,v} = 1 \iff z_{r,c} = v$ cho cùng tập nghiệm Sudoku như biểu diễn SAT, dù cách solver biểu diễn nội bộ khác nhau.

7. Thuật toán và triển khai adapter

Giả mã dưới đây tóm tắt luồng đã triển khai bằng parser, model builder, solver adapter, decoder và checker riêng. Các công thức ở phần trước là đặc tả mô hình; source code thực nằm trong repository. Giả mã không thay cho runner có hard process guard và không chứng minh một đợt benchmark đã chạy.

Giả mã: giải một input

1. Đọc/parse input thành Puzzle; lỗi cấu trúc, JSON hoặc file trả INVALID_INPUT. Kiểm method/encoding/budget theo API trước khi dựng mô hình.
2. Chuẩn bị thư viện backend ngoài phần đo; runtime thiếu được báo UNAVAILABLE.
3. Bắt đầu đồng hồ đơn điệu t_0; lấy deadline sớm hơn giữa t_0+cutoff và deadline caller (nếu có). Nếu hết ngân sách thì TIMEOUT.
4. Dựng CNF hoặc model CP/ILP và tạo/nạp solver mới. Tính phần build/setup; nếu đã hết ngân sách thì bỏ qua solve và trả TIMEOUT.
5. Gọi solver với ngân sách còn lại, dùng giới hạn native và cơ chế ngắt hợp tác được API hỗ trợ. Giữ raw status.
6. Chốt thời điểm kết thúc native solve và các thời gian đã đo. Kết quả đến muộn thành TIMEOUT; không nhận incumbent đến muộn.
7. Nếu chứng minh không khả thi, trả UNSAT. Nếu không có candidate hợp lệ, phân loại TIMEOUT, ERROR hoặc LICENSE_LIMIT theo bằng chứng; không biến unknown thành UNSAT.
8. Giải mã candidate thành bảng đầy đủ; lỗi decode là ERROR. SAT chỉ giải mã biến chính; MIP phải qua kiểm tính hữu hạn/nguyên trước khi chọn giá trị.
9. Chạy checker độc lập kiểm miền, hàng, cột, khối và clues. Chỉ trả SAT kèm nghiệm nếu checker chấp nhận; từ chối hoặc lỗi checker là ERROR.
10. Trên mọi đường thoát, dừng/join timer trước khi hủy native solver và dọn tài nguyên. Lỗi cleanup được báo rõ; không công bố nghiệm thành công khi cleanup lỗi.

Hình 2: Luồng adapter, không phải bằng chứng hard cutoff của runner.

7.1. Cách gọi thư viện và kiểm nghiệm

Backend SAT là `pysat.solvers.Glucose3` trong python-sat 1.9.dev16 [10, 1, 2]. Năm đối chứng dùng OR-Tools CP-SAT [4], gurobipy [5], CPLEX Python API trực tiếp (`cplex.Cplex`, không dùng DOcplex.MP) [8], DOcplex.CP cho CP Optimizer [9], và highspy cho HiGHS [6, 7]. SciPy chỉ cung cấp ma trận sparse cho ILP trong các adapter này, không phải backend solve HiGHS qua `scipy.optimize.milp`. Phía SAT, API công khai là `prepare_backend` và `solve_sat`; mỗi lượt dùng Glucose3 mới. Adapter gọi CNF builder với encoding đã chọn, dùng native solve có hỗ trợ interrupt, rồi chuyển model qua decoder và `validation.check_solution`. Decoder không suy giá trị ô từ biến phụ.

Phía đối chứng, `solve_baseline` nhận một trong các method `cpsat`, `gurobi`, `cplex_mip`, `cpoptimizer`, `highs`. CP-SAT/CP Optimizer dùng builder CP; Gurobi/CPLEX MIP/HiGHS nhận cùng ma trận ILP, bounds, integrality và objective hằng. Mỗi lượt có đối tượng mới và một luồng/worker. Các adapter giữ raw status và dùng cùng checker production.

MIP chỉ chấp nhận giá trị hữu hạn cách 0 hoặc 1 không quá 10^{-6}, với đúng một giá trị được chọn mỗi ô; không làm tròn tùy ý một nghiệm phân số. HiGHS còn phải báo primal feasibility/integrality phù hợp trước decode. Một incumbent trong lượt time-limit chỉ có thể thành SAT nếu đến đúng hạn và checker chấp nhận. UNSAT cần bằng chứng infeasibility; thiếu runtime, vượt license và unknown được phân biệt.

7.2. Ranh giới thời gian và bảo vệ tiến trình

Imports/đọc input nằm ngoài phần đo. Build gồm dựng mô hình/CNF, tạo và nạp solver; solve dùng phần ngân sách còn lại. Decode, checker và cleanup nằm ngoài build+solve. Việc nhận nghiệm theo deadline dựa vào thời điểm native solve kết thúc, không dùng thời điểm checker hoàn tất để tính thời gian solve. Các thành phần thời gian không biết sau khi bị dừng cưỡng bức phải để thiếu, không bịa số.

Ngắt native của SAT và timer/giới hạn native của baseline là hợp tác. Riêng DOcplex 2.32.264, TimeLimit nằm trong CPO đã gửi, nên không hứa chỉnh chính xác native TimeLimit sau load: adapter dùng preload để tính build, timer gọi abort_search theo deadline thực và từ chối kết quả muộn. Preload hook riêng cần kiểm lại khi nâng DOcplex. Nếu timer join thất bại, adapter báo ERROR và có thể hoãn hủy native để tránh callback vào bộ nhớ đã giải phóng.

Measurement-end hook và hard worker/process-tree guard đã được nghiệm thu. Runner dùng START/END để áp cutoff đúng phần đo, rồi chuyển sang watchdog hậu xử lý và dọn process tree khi cần. Pilot đã kiểm đường tích hợp này; native timeout riêng vẫn là hợp tác. Mục 8 mô tả cấu hình đợt chính đã khóa; Mục 9 báo kết quả thực chạy, tách khỏi pilot.

8. Thiết lập thực nghiệm

Đợt chính `main-20261016-01` đã thực hiện đủ 906 lượt mới với cutoff 10 giây. Dataset, source code, cấu hình và dependency đã khóa; `run.json` và `execution-freeze.json` lưu metadata/hash, được đối chiếu với audit analysis final. Các kết quả ở Mục 9 chỉ thuộc đợt chính này.

8.1. Máy và runtime

Các lượt được cấu hình chạy tuần tự trên cùng máy Windows x64, một job tại một thời điểm và một luồng/worker mỗi engine. Bảng 2 tách binding khỏi native runtime khi có bằng chứng. Không suy phiên bản native Glucose3 từ PySAT, hoặc HiGHS nhúng SciPy từ highspy.

Bảng 2
Máy và phiên bản trong metadata đợt chính đã khóa.

Thành phần	Giá trị đã ghi
OS	Windows 11, 10.0.26300, x64
CPU	AMD Ryzen 7 H 260 w/ Radeon 780M Graphics; 16 logical CPUs
RAM	33,618,251,776 bytes (≈ 31.31 GiB)
Python	3.13.5
SAT	python-sat 1.9.dev16; Glucose3 (probe không có API version native riêng)
CP-SAT	OR-Tools 9.15.6755
HiGHS	highspy 1.15.1; native 1.15.1
Gurobi	gurobipy 13.0.3; native 13.0.3
CPLEX MIP	cplex 22.2.0.1; native 22.2.0.0
CP Optimizer	docplex 2.32.264; native 22.2.0.0
Ma trận sparse	SciPy 1.18.1

8.2. Dữ liệu và tập so sánh

S80 gồm 60 synthetic: bốn kích thước $N = 4, 9, 16, 25$, ba mật độ clue mục tiêu 30%, 50%, 70%, mỗi nhóm năm đề; cộng 20 public 9×9. Clue count và seed thực dùng nằm trong manifest đã khóa. Public lấy từ một file hard.txt của Sudoku Exchange, commit `d8c8ebaee0c08c412cfba96af1923dfa61c83317` [15]; seed lấy mẫu là 20261016, giữ thứ tự mẫu đã lưu.

Chọn B16 trước kết quả: trong mỗi nhóm synthetic kích thước–mật độ, sắp theo actual seed rồi instance ID và lấy đề đầu (12 đề), cộng bốn public đầu theo thứ tự mẫu cố định. C10 là phần $4 \times 4/9 \times 9$ của B16: ba synthetic mỗi kích thước và cùng bốn public. Có $C10 \subset B16 \subset S80$. SAT chạy ba encoding trên S80; CP-SAT/HiGHS dùng B16; Gurobi/CPLEX MIP/CP Optimizer dùng C10.

Không chạy lại SAT riêng cho các tập con. Cấu hình chính có ba lần lặp mỗi cặp đề–cấu hình: 720 + 96 + 90 = 906 lượt, schedule xáo bằng seed 20261016 và lưu trước chạy. Pilot đã thực thi 55 job ở lần lặp thứ nhất để kiểm tích hợp/độ phủ, không phải 906 lượt hoặc ba lần lặp chính. Không dùng tỷ lệ SAT/timeout hay thời gian pilot làm Results chính.

8.3. Đo thời gian và bảo vệ tiến trình

Worker import/đọc input trước START. START/END dùng trạng thái chia sẻ đồng bộ và đồng hồ đơn điệu. Từ START, cutoff 10 giây gồm dựng mô hình/CNF, tạo/nạp solver và solve; solve chỉ dùng ngân sách còn lại. END đúng hạn chuyển sang watchdog hậu xử lý riêng. Decode, validation, cleanup và ghi CSV/I/O đầu ra nằm ngoài build+solve. Thiếu hoặc muộn END là TIMEOUT; các thành phần thời gian chưa biết được để thiếu.

Runner Windows gán worker vào kill-on-close Job Object trước khi cho chạy, dừng process tree khi quá hạn và chờ hết descendants. Startup watchdog là 60 giây trước START; hậu xử lý là 30 giây sau END đúng hạn. Lỗi startup/hậu xử lý/cleanup là ERROR, không trộn với timeout đo. Watchdog không cấp thêm thời gian solve. Hard guard và measurement-end hook đã qua pilot trước khi dùng cho 906 lượt chính mới.

CP Optimizer dùng private preload của DOcplex 2.32.264 để tính compile/send/load vào build. Native TimeLimit nhúng trong CPO trước load; sau load, timer abort_search hợp tác được bổ sung bởi external guard. Không hứa chỉnh chính xác native TimeLimit sau load; nâng DOcplex cần kiểm lại private hook như Mục 7.

8.4. Quy tắc tổng hợp

Với mỗi đề–cấu hình đủ ba SAT, lấy median ba lần lặp cho từng thành phần build, solve và total; sau đó lấy median các median theo đề trong đúng nhóm. Ba thành phần được tổng hợp độc lập: ở mức mỗi lượt total bằng build+solve (setup nằm trong build), nhưng hai median nhóm không nhất thiết cộng thành median total. So sánh runtime cặp chỉ dùng ID đủ ba SAT ở cả hai bên, kèm matched count; solved count và timeout vẫn công bố trên toàn tập được chọn. PAR-2 theo lượt là total của SAT và $2 \times 10 = 20$ giây cho TIMEOUT; ERROR, INVALID_INPUT, UNAVAILABLE, LICENSE_LIMIT bị loại khỏi penalty. Không lấy lượt nhanh nhất, không điền timing thiếu bằng 0.

9. Kết quả và thảo luận

Phân tích dùng duy nhất đợt chính `main-20261016-01`: 906 lượt, 903 SAT qua checker độc lập và 3 TIMEOUT; không có UNSAT, ERROR, INVALID_INPUT, LICENSE_LIMIT hoặc UNAVAILABLE. Pilot không được gộp vào số liệu. Bảng 3 ghi cả số lượt và số đề đủ ba SAT trong đúng phạm vi.

Bảng 3
Độ phủ của 906 lượt chính.

Cấu hình	Tập	SAT/lượt	Đề đủ 3 SAT	TIMEOUT
Glucose3 pairwise	S80	240/240	80/80	0
Glucose3 sequential	S80	240/240	80/80	0
Glucose3 bitwise	S80	240/240	80/80	0
CP-SAT	B16	48/48	16/16	0
HiGHS ILP	B16	45/48	15/16	3
Gurobi MIP	C10	30/30	10/10	0
CPLEX MIP	C10	30/30	10/10	0
CP Optimizer	C10	30/30	10/10	0

9.1. Kích thước CNF và chi phí dựng mô hình

Hình 3 giữ mật độ 50% để tách ảnh hưởng của kích thước bảng; trục tung log, số biến gồm cả biến phụ, count lấy trước preprocessing. Hình dùng seed 0 đại diện vì count khớp giữa năm seed trong mỗi nhóm kích thước–mật độ. Bảng 4 minh họa riêng N25/30% với 188 clues: pairwise không có biến phụ nhưng nhiều clause nhất; sequential có nhiều biến phụ hơn bitwise nhưng ít clause hơn. Count CNF không được dùng thay kích thước native CP/MIP.

Hình 3: Kích thước CNF synthetic, mật độ clue cố định 50%; seed 0 mỗi kích thước, count trước preprocessing.

Bảng 4
CNF thực của synthetic-N25-p30-s0, 188 clues; trước preprocessing.

Encoding	Biến chính	Biến phụ	Tổng biến	Clauses
pairwise	15625	0	15625	752688
sequential	15625	60000	75625	180188
bitwise	15625	12500	28125	315188

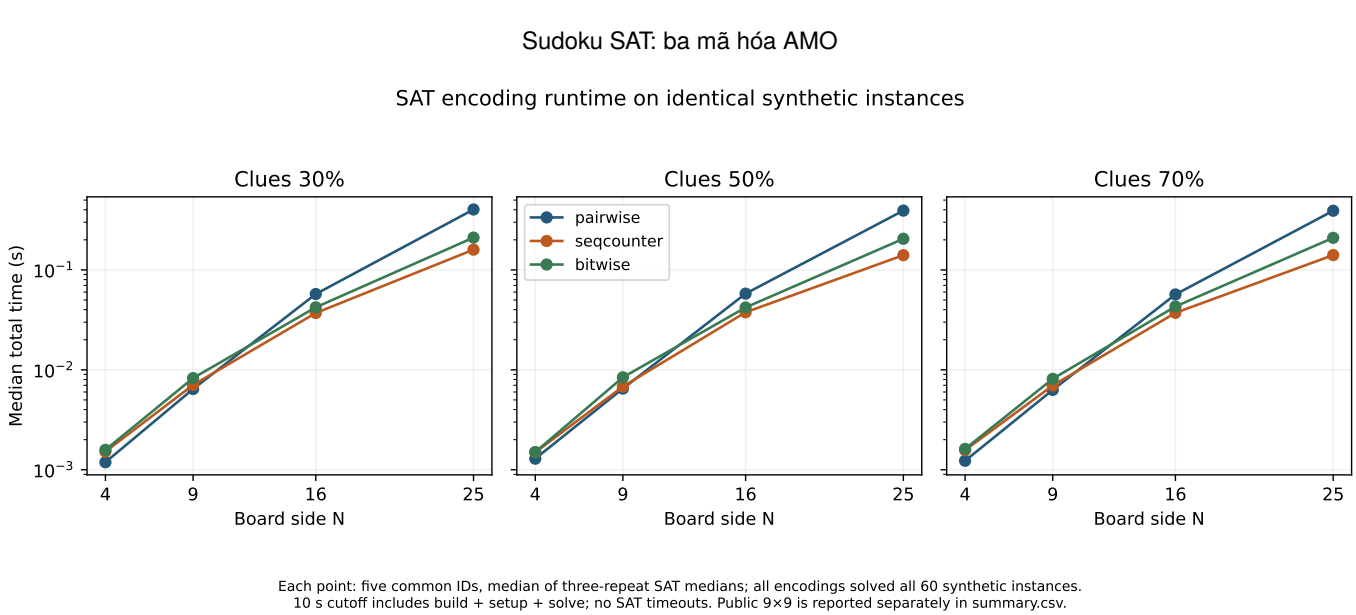
Bảng 5 dùng cùng năm ID `synthetic-N25-p30-s0` đến `s4`, mỗi ID đủ ba SAT ở cả ba encoding. Build (gồm tạo/nạp solver) lớn hơn solve trong nhóm này. Sequential có median total thấp nhất, dù median solve của bitwise thấp hơn sequential. Vì vậy, chỉ xét solve hoặc số biến sẽ bỏ qua chi phí quan trọng của pipeline.

Bảng 5
Synthetic 25×25, 30% clues: năm ID chung, giây.

Encoding	Median build	Median solve	Median total
pairwise	0.381882	0.022380	0.402449
sequential	0.130810	0.024383	0.159527
bitwise	0.193860	0.015416	0.210650

Mỗi cột là median của median ba lượt theo đề, tổng hợp độc lập; không cộng hai median nhóm để suy median total.

Hình 4 giữ riêng ba mật độ; mỗi điểm là median của năm median theo đề, mỗi median đề từ ba lần lặp. Thứ tự không cố định: ở synthetic 4×4/30%, median total pairwise/sequential/bitwise lần lượt là 0.001189/0.001520/0.001582 giây. Sequential thấp nhất ở N25/30% không đồng nghĩa thấp nhất cho mọi n hoặc mọi dataset.



Hình 4: Median total SAT (giây), cùng năm synthetic ID mỗi điểm; ba lần lặp/ID, cutoff 10 giây gồm build (đã gồm setup) + solve. Ba encoding không có timeout.

9.2. Đối chứng trên đúng tập giao

Các Bảng 6–8 lấy từ `paper/baseline-matched.csv`, dùng pairwise làm mốc cố định, không chọn encoding nhanh nhất riêng cho từng đối thủ. Median total chỉ lấy ID có đủ ba SAT ở cả hai bên. Trên B16, mỗi mật độ tại N25 chỉ có một ID, khác năm ID của thống kê S80 trong Bảng 5. Ví dụ `synthetic-N25-p30-s0`: CP-SAT 0.230552 giây so với pairwise 0.394866 giây; không thay giá trị pairwise này bằng 0.402449 giây của nhóm năm ID.

Bảng 6
B16 synthetic: median total trên từng ID chung, giây.

N	Clues (%)	m_{CP}/m_H	CP-SAT	HiGHS	Pairwise
4	30	1/1	0.004866	0.001704	0.001184
4	50	1/1	0.004223	0.001638	0.001453
4	70	1/1	0.004099	0.001468	0.001323
9	30	1/1	0.013557	0.012148	0.006350
9	50	1/1	0.006484	0.004400	0.006544
9	70	1/1	0.005491	0.004467	0.006700
16	30	1/1	0.057879	0.047280	0.057086
16	50	1/1	0.009205	0.021862	0.058682
16	70	1/1	0.007168	0.022792	0.055755
25	30	1/0	0.230552	–	0.394866
25	50	1/1	0.055806	0.113672	0.382179
25	70	1/1	0.012724	0.066873	0.383525

m là số ID đủ ba SAT ở cả hai bên. Mỗi hàng chỉ có một ID, khác năm ID/hàng của S80. Dấu –: HiGHS có 3 TIMEOUT ở N25/30%, không có median SAT.

Bảng 7
C10 synthetic: một ID chung mỗi hàng, median total (giây).

N	Clues (%)	m	Gurobi	CPLEX MIP	CP Optimizer	Pairwise
4	30	1	0.002521	0.002137	0.029427	0.001184
4	50	1	0.002477	0.002280	0.033032	0.001453
4	70	1	0.002210	0.002362	0.030086	0.001323
9	30	1	0.010647	0.006728	0.029006	0.006350
9	50	1	0.007375	0.003787	0.029938	0.006544
9	70	1	0.007339	0.003884	0.030118	0.006700

Bảng 8
Bốn public 9×9 chung của B16/C10, median total (giây).

Backend	Tập	m	Backend	Pairwise
CP-SAT	B16	4	0.006506	0.006534
HiGHS ILP	B16	4	0.004704	0.006534
Gurobi MIP	C10	4	0.008019	0.006534
CPLEX MIP	C10	4	0.004748	0.006534
CP Optimizer	C10	4	0.030911	0.006534

Cùng bốn ID L069940, L293964, L269904, L215069 (prefix public-se-hard-). Pairwise là mốc cố định; không chọn encoding nhanh nhất cho từng đối thủ.

Trên cùng bốn public, CPLEX MIP có median total 0.004748 giây, thấp hơn pairwise 0.006534 giây; Gurobi 0.008019 và CP Optimizer 0.030911 giây cao hơn pairwise. Đây là ví dụ chống nhận định SAT luôn thắng, không phải bằng chứng ILP hoặc một solver luôn tốt nhất. Các bảng không báo ratio; nếu cần ratio theo cặp phải lấy median của ratio từng ID, không chia hai median nhóm. Chênh lệch millisecond trên tập nhỏ cần được diễn giải thận trọng.

9.3. Timeout, PAR-2 và truy vết

Cả ba timeout là ba lần lặp HiGHS của `synthetic-N25-p30-s0`. ID này không có median SAT; tập giao runtime HiGHS với SAT/CP-SAT trên B16 còn 15 ID, còn với thương mại trên C10 vẫn 10 ID. Hai encoding SAT có 80 ID chung. Mỗi timeout nhận PAR-2 = $2 \times 10 = 20$ giây, tổng ba penalty là 60 giây; mean PAR-2 của đúng nhóm HiGHS N25/30% là 20 giây. Không coi timing thiếu là 0: r3 thiếu cả build/solve/total vẫn để thiếu, r1/r2 có observed total khoảng cutoff nhưng vẫn TIMEOUT. Runtime của phần giải được luôn phải đọc cùng count trong Bảng 3. Các bảng/hình lấy từ analysis final đã khóa; `summary.csv`, `instances.csv`, `matched-pairs.csv` và `cnf.csv` giữ thống kê chi tiết. `paper/raw-row-index.csv` nối job ID tới dòng CSV gốc; audit giữ config, phiên bản và hash. Không gộp pilot, không chạy lại benchmark để chọn kết quả thuận lợi.

10. Giới hạn nghiên cứu

Synthetic xuất phát từ bảng hợp lệ và biến thể có cấu trúc nên có generator bias, không đại diện mọi Sudoku. Lớn hơn hoặc ít clue hơn không tự chứng minh khó hơn. Public cũng do nguồn ngoài sinh bằng máy, chỉ chọn 20 đề từ một bucket hard.txt; bucket/rating là nhãn nguồn, chưa được dự án hiệu chuẩn. Bộ 80 đề có evidence SAT nhưng uniqueness chưa kiểm; tìm một nghiệm hợp lệ không phải chứng nhận duy nhất.

Đối chứng là mẫu bổ sung nhỏ: so ba SAT trên S80; với CP-SAT/HiGHS phải lọc B16; với thương mại phải lọc C10 hoặc tập giao khả dụng. So runtime cặp dùng đề có đủ ba lượt thành công ở cả hai cấu hình, công bố số đề/độ phủ đúng phạm vi. Không ghép median từ hai tập khác nhau như cùng mẫu. Timeout 10 giây là dữ liệu bị chặn theo cutoff, không phải thời gian giải thực. ERROR, UNAVAILABLE và LICENSE_LIMIT tách khỏi TIMEOUT và không tính penalty PAR-2 của timeout.

Bản miễn phí giới hạn phạm vi: Gurobi size-limited có ngưỡng 2.000 biến/2.000 ràng buộc tuyến tính; CPLEX MIP Community là 1.000/1.000. Các mô hình $4 \times 4/9 \times 9$ của C10 nằm trong ngưỡng và đã chạy đủ ba lần lặp chính trên C10; không suy độ phủ toàn S80. CP Optimizer có giới hạn riêng phụ thuộc mô hình/không gian tìm kiếm: lỗi size trên một bảng 16×16 hoặc 25×25 không chứng minh mọi bảng cùng cỡ đều không chạy được. HiGHS không được đổi nhãn thành Gurobi/CPLEX.

Một máy, một cấu hình phiên bản, ba lần lặp và cutoff hữu hạn chưa đủ khái quát cho mọi phần cứng/solver. B16 chỉ có một ID mỗi synthetic stratum; C10 chỉ có bảng nhỏ và bốn public chung. Chênh lệch millisecond dễ chịu ảnh hưởng overhead; chưa có kiểm định thống kê, chưa đo RAM hoặc search counters. Các đối chứng dùng mô hình và engine khác SAT nên không quy mọi chênh lệch cho encoding. Smoke/pilot chỉ là gate tích hợp riêng, không thuộc kết quả chính.

11. Kết luận

Mô hình ô–giá trị với bốn họ EXO cho phép thay AMO mà giữ đặc tả Sudoku chung. Ba encoding pairwise, sequential counter và bitwise cùng Glucose3 giải đủ 80 đề ở cả ba lần lặp. Trên năm synthetic N25/30%, sequential có median total 0.159527 giây so với pairwise 0.402449 và bitwise 0.210650 giây; build là phần lớn chi phí trong nhóm này. Kết quả trả lời câu hỏi về encoding bằng tradeoff biến/clause và cả chi phí dựng/nạp, không chỉ solve. Thứ tự thay đổi ở bảng nhỏ nên không chọn một encoding thắng phổ quát. CP-SAT/HiGHS trên B16 và các solver thương mại trên C10 cung cấp đối chứng có phạm vi: HiGHS timeout cả ba lượt của một ID; CPLEX MIP có median thấp hơn pairwise trên bốn public chung. Không suy SAT, CP hoặc ILP luôn thắng từ các mẫu nhỏ này. Nghiên cứu tiếp theo nên mở rộng nguồn đề, kiểm uniqueness và độ khó, tăng số lần lặp/máy, và tách thêm chi phí runtime; cần giữ matched subsets cùng cách xử lý dữ liệu bị cutoff.

Khả dụng mã nguồn và dữ liệu

Mã nguồn, cấu hình, bộ dữ liệu, CSV kết quả, metadata freeze và analysis được kèm trong gói bàn giao cục bộ. Đợt chính là `results/raw/main-20261016-01`; bảng/hình và audit ở `results/summary/main-20261016-01`. Dữ liệu public giữ provenance/license Sudoku Exchange [15]; notices của các thành phần tái dùng nằm trong gói nguồn. Báo cáo dùng template cas-sc và Tectonic [3, 16]. Repository đích do người dùng cung cấp là `https://github.com/Ngynkybyrd/2627I_INT3011E`. Nội dung dự án hiện ở gói cục bộ, chưa được đồng bộ lên repository đích; chưa xác nhận mã nguồn hoặc kết quả đã khả dụng trực tuyến. Gói chưa được công bố hoặc nộp.

Tài liệu tham khảo

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the Twenty-First International Joint Conference on Artificial Intelligence, pp. 399–404. URL: https://www.ijcai.org/Proceedings/09/Papers/074.pdf.

[2] Eén, N., Sörensson, N., 2004. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing, Springer. pp. 502–518. URL: https://doi.org/10.1007/978-3-540-24605-3_37, doi:10.1007/978-3-540-24605-3_37. proceedings of SAT 2003; publisher citation year 2004.

[3] Elsevier, 2024. Elsevier CAS Templates. Elsevier. URL: https://ctan.org/pkg/els-cas-templates. version 2.4; template provenance, not a research reference.

[4] Google OR-Tools Team, n.d. CP-SAT Solver. Google. URL: https://developers.google.com/optimization/cp/cp_solver. living documentation; accessed 16 October 2026. Recorded OR-Tools distribution: 9.15.6755.

[5] Gurobi Optimization, LLC, 2026. Gurobi Optimizer Reference Manual. URL: https://docs.gurobi.com/projects/optimizer/en/current/reference/python/model.html. year follows the vendor's explicit 2026 citation recommendation; accessed 16 October 2026. Recorded gurobipy and native runtime: 13.0.3.

[6] HiGHS Developers, n.d. HiGHS Documentation: Python Examples. URL: https://ergo-code.github.io/HiGHS/dev/interfaces/python/example-py/. living documentation; accessed 16 October 2026. The recorded local highspy runtime is 1.15.1.

[7] Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142. URL: https://doi.org/10.1007/s12532-017-0130-5, doi:10.1007/s12532-017-0130-5. preferred citation recommended by the HiGHS project; not a description of its full MIP algorithm.

[8] IBM, n.d.a. CPLEX Python API Reference Manual: Meet the Python API. URL: https://ibmdecisionoptimization.github.io/cplex-python-doc/manual/03_meet_python.html. documentation banner 22.1.2; accessed 16 October 2026. API reference only: the installed cplex distribution is 22.2.0.1 and recorded native runtime is 22.2.0.0; not DOcplex.MP.

[9] IBM, n.d.b. Creating a Constraint Programming Model (DOcplex.CP). URL: https://ibmdecisionoptimization.github.io/docplex-doc/cp/creating_model.html. documentation banner V2.32; accessed 16 October 2026. Recorded DOcplex interface: 2.32.264; CP Optimizer executable runtime: 22.2.0.0.

[10] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26, doi:10.1007/978-3-319-94144-8_26.

[11] Lynce, I., Ouaknine, J., 2006. Sudoku as a SAT problem. Author-hosted paper, listed in Proceedings of AIMATH 06. URL: https://people.mpi-sws.org/~joel/publications/sudoku05.pdf. 9-page author version; author landing page lists AIMATH 06 (2006), with copyright 2005.

[12] PySAT Developers, n.d.a. Cardinality Encodings (pysat.card). PySAT. URL: https://pysathq.github.io/docs/html/api/card.html. documentation banner 1.9.dev16; accessed 16 October 2026.

[13] PySAT Developers, n.d.b. PySAT cardinality encoder source: bitwise.hh. URL: https://github.com/pysathq/pysat/blob/218c1c163f837c6512d8970e4bdf49ae02cee779/cardenc/bitwise.hh. upstream commit 218c1c163f837c6512d8970e4bdf49ae02cee779; header credits the card library from BOLT; accessed 16 October 2026. Not yet matched to the installed wheel.

[14] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. URL: https://www.carstensinz.de/papers/CP-2005.pdf, doi:10.1007/11564751_73.

[15] Sudoku Exchange, n.d. Sudoku Exchange Puzzle Bank: hard.txt. GitHub repository grantm/sudoku-exchange-puzzle-bank. URL: https://github.com/grantm/sudoku-exchange-puzzle-bank/tree/d8c8ebaee0c08c412cfba96af1923dfa61c83317. commit d8c8ebaee0c08c412cfba96af1923dfa61c83317; dataset public-domain dedication; accessed 16 October 2026. Uniqueness is a source claim, not independently verified.

[16] Tectonic Project, n.d. The Tectonic Typesetting System: Installation. Tectonic Project. URL: https://tectonic-typesetting.github.io/book/latest/installation/. accessed 16 October 2026; compiler release 0.17.0.